

SE351 Programming Fundamentals

Dr. Farhan Ahmed Siddiqui
farhan@uok.edu.pk

Course Objective

This course serves as an introduction to computational problem solving using a modern programming language. The main objective is to provide students with understanding of programming essentials, including algorithms, data types, elementary control structures, functions, and recursion used within the framework of structural and modular programming paradigms. The students will learn to apply an integrated program development tool to design, implement, test, debug, and document programs. This course prepares students for the more advanced programming courses such as Object-Oriented Programming Techniques and Data Structures.

What is Computer Science?

- NOT about coding or hardware or software!
- Computer Science is about PROBLEM SOLVING
- Computer Science is about ***DEVELOPING ALGORITHMS*** to solve complex problems

Defining Computing Discipline

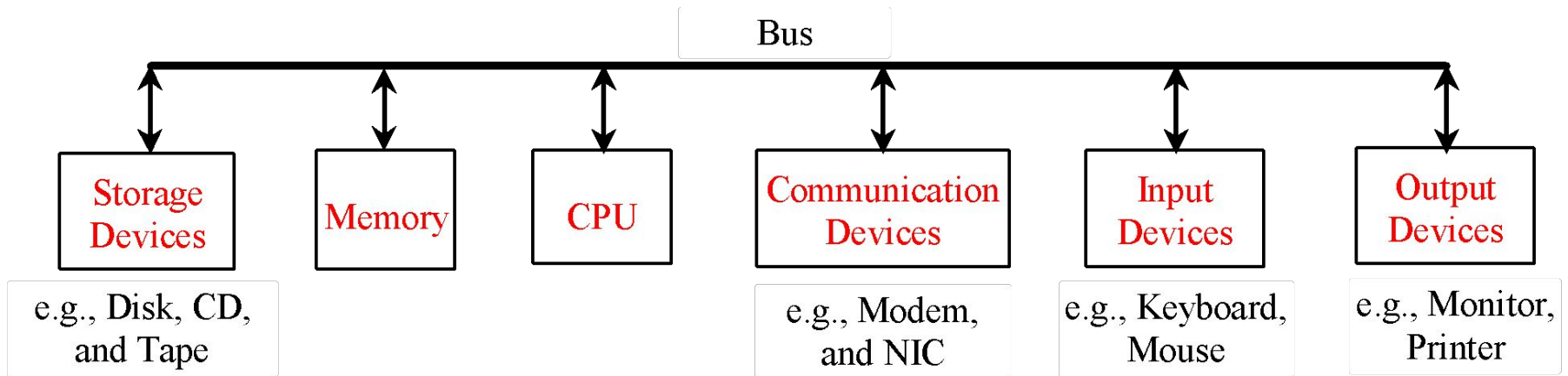
- The discipline of computing is the systematic study of algorithmic processes that describe and transform information; their theory, analysis, design, efficiency, implementation, and application. The fundamental question underlying all of computing is, “What can be (efficiently) automated?”

What is Software Engineering?

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

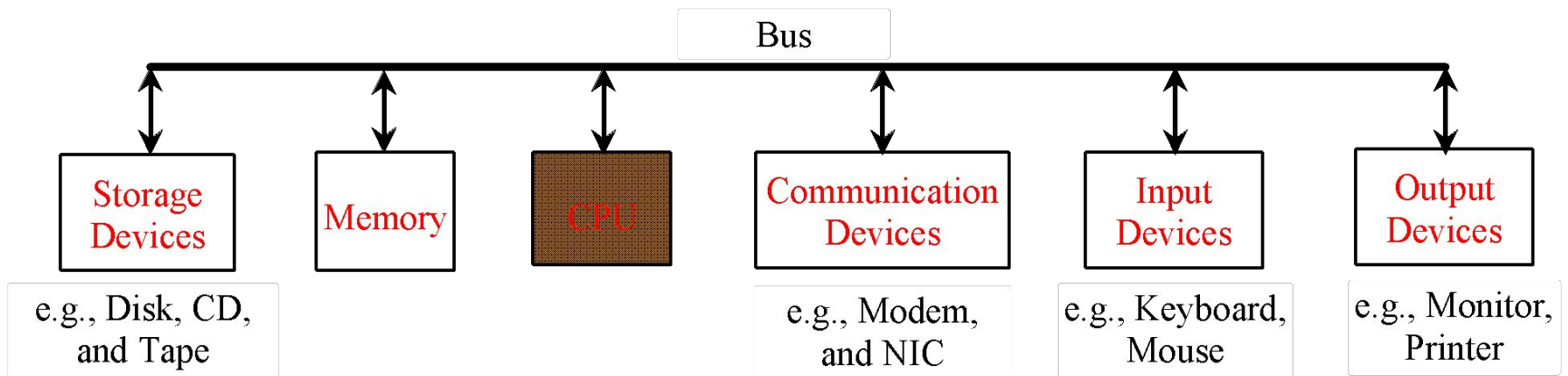
What is a Computer?

A computer consists of a CPU, memory, hard disk, floppy disk, monitor, printer, and communication devices.



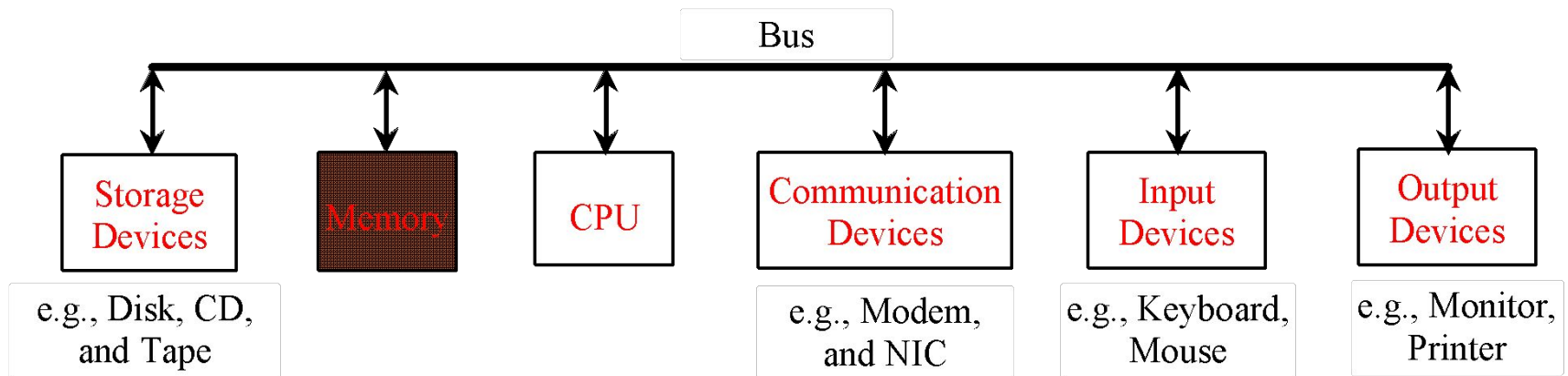
CPU

The central processing unit (CPU) is the brain of a computer. It retrieves instructions from memory and executes them. The CPU speed is measured in megahertz (MHz) / GHz, with 1 megahertz equaling 1 million pulses per second. The speed of the CPU has been improved continuously. If you buy a PC now, you can get an Intel i9 Processor at 6 GHz (1 GHz is 1000 MHz).



Memory

Memory is to store data and program instructions for CPU to execute. A memory unit is an ordered sequence of bytes, each holds eight bits. A program and its data must be brought to memory before they can be executed. A memory byte is never empty, but its initial content may be meaningless to your program. The current content of a memory byte is lost whenever new information is placed in it.



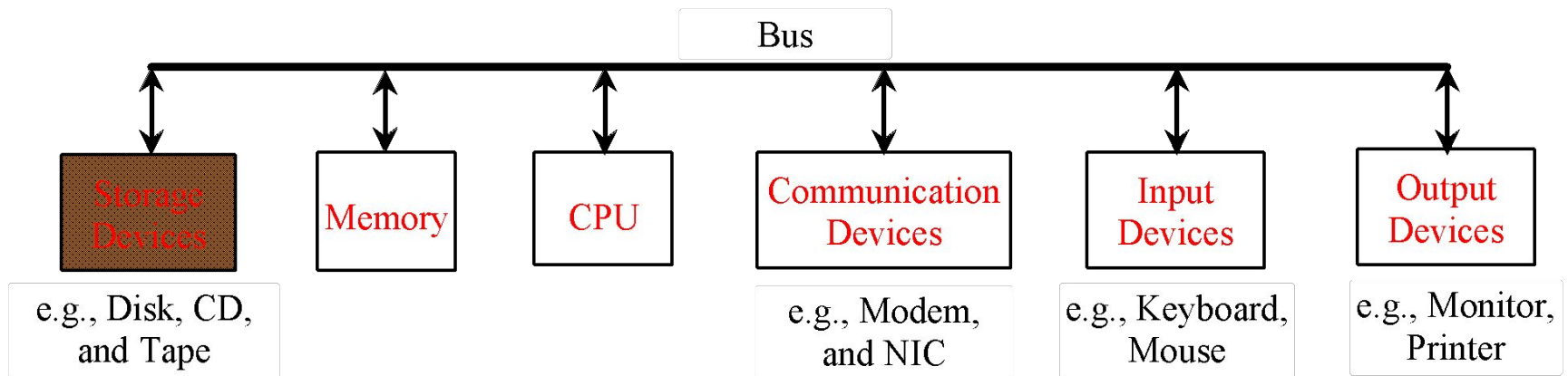
How Data is Stored?

Data of various kinds, such as numbers, characters, and strings, are encoded as a series of bits (zeros and ones). Computers use zeros and ones because digital devices have two stable states, which are referred to as *zero* and *one* by convention. The programmers need not to be concerned about the encoding and decoding of data, which is performed automatically by the system based on the encoding scheme. The encoding scheme varies. For example, character 'J' is represented by 01001010 in one byte. A small number such as three can be stored in a single byte. If computer needs to store a large number that cannot fit into a single byte, it uses a number of adjacent bytes. No two data can share or split a same byte. A byte is the minimum storage unit.

Memory address	Memory content	
.	.	
.	.	
.	.	
2000	01001010	Encoding for character 'J'
2001	01100001	Encoding for character 'a'
2002	01110110	Encoding for character 'v'
2003	01100001	Encoding for character 'a'
2004	00000011	Encoding for number 3

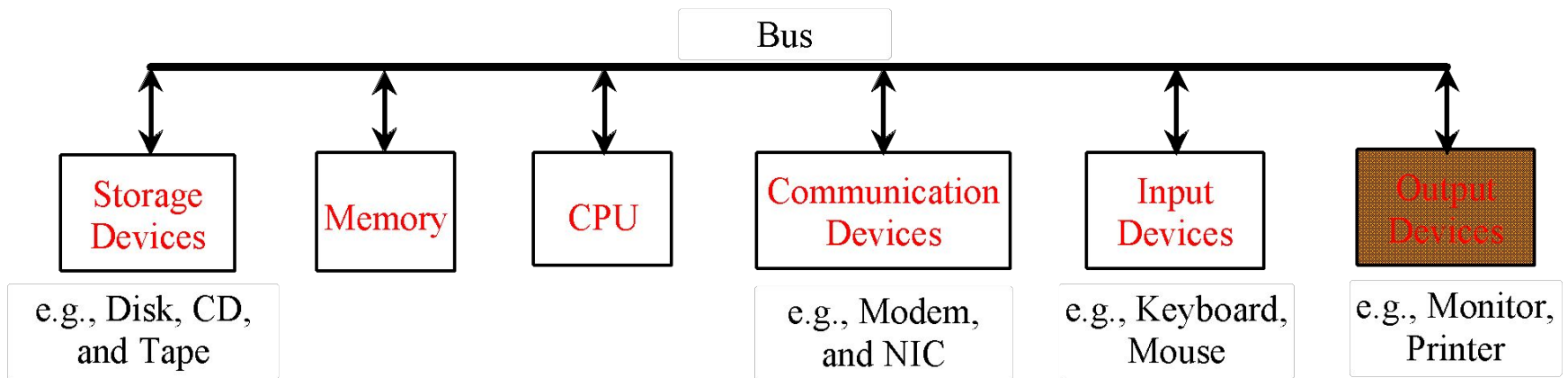
Storage Devices

Memory is volatile, because information is lost when the power is off. Programs and data are permanently stored on storage devices and are moved to memory when the computer actually uses them. There are three main types of storage devices: Disk drives (hard disks and floppy disks), CD drives (CD-R and CD-RW), and Tape drives.



Output Devices: Monitor

The monitor displays information (text and graphics). The resolution and dot pitch determine the quality of the display.



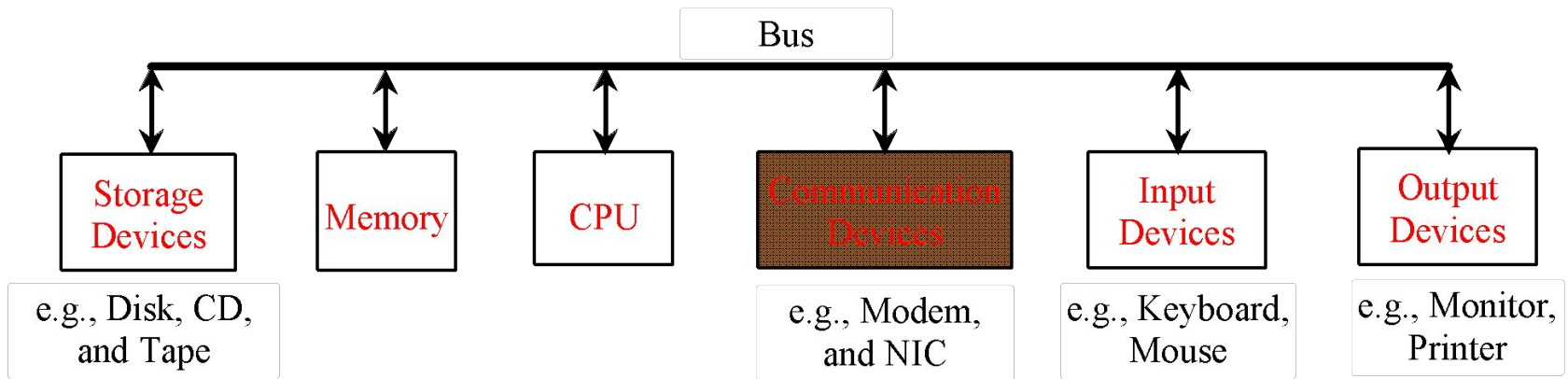
Monitor Resolution and Dot Pitch

resolution The *screen resolution* specifies the number of pixels in horizontal and vertical dimensions of the display device. *Pixels* (short for “picture elements”) are tiny dots that form an image on the screen. A common resolution for a 17-inch screen, for example, is 1,024 pixels wide and 768 pixels high. The resolution can be set manually. The higher the resolution, the sharper and clearer the image is.

dot pitch The *dot pitch* is the amount of space between pixels, measured in millimeters. The smaller the dot pitch, the sharper the display.

Communication Devices

A *regular modem* uses a phone line and can transfer data in a speed up to 56,000 bps (bits per second). A *DSL* (digital subscriber line) also uses a phone line and can transfer data in a speed 20 times faster than a regular modem. A *cable modem* uses the TV cable line maintained by the cable company. A cable modem is as fast as a DSL. Network interface card (*NIC*) is a device to connect a computer to a local area network (LAN). The LAN is commonly used in business, universities, and government organizations. A typical type of NIC, called *10BaseT*, can transfer data at 10 mbps (million bits per second).



Programs

Computer *programs*, known as *software*, are instructions to the computer.

You tell a computer what to do through programs. Without programs, a computer is an empty machine. Computers do not understand human languages, so you need to use computer languages to communicate with them.

Programs are written using programming languages – **is a formal artificial language with a set of rules (grammar) that describes how the elements of the language are used together (syntax)**

Programming Languages

Machine Language Assembly Language High-Level Language

Machine language is a set of primitive instructions built into every computer. The instructions are in the form of binary code, so you have to enter binary codes for various instructions. Program with native machine language is a tedious process. Moreover the programs are highly difficult to read and modify. For example, to add two numbers, you might write an instruction in binary like this:

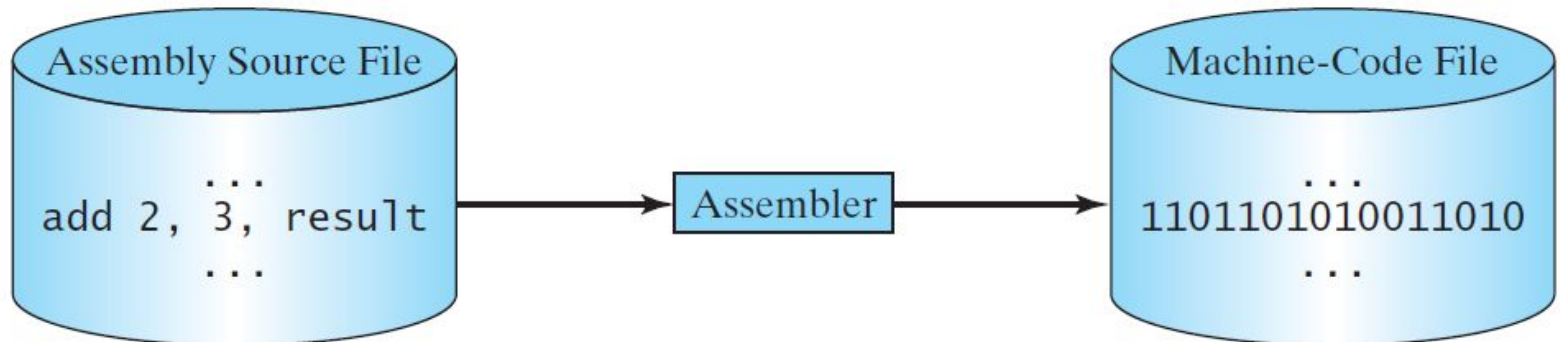
```
1101101010011010
```

Programming Languages

Machine Language **Assembly Language** High-Level Language

Assembly languages were developed to make programming easy. Since the computer cannot understand assembly language, however, a program called assembler is used to convert assembly language programs into machine code. For example, to add two numbers, you might write an instruction in assembly code like this:

ADDF3 R1, R2, R3



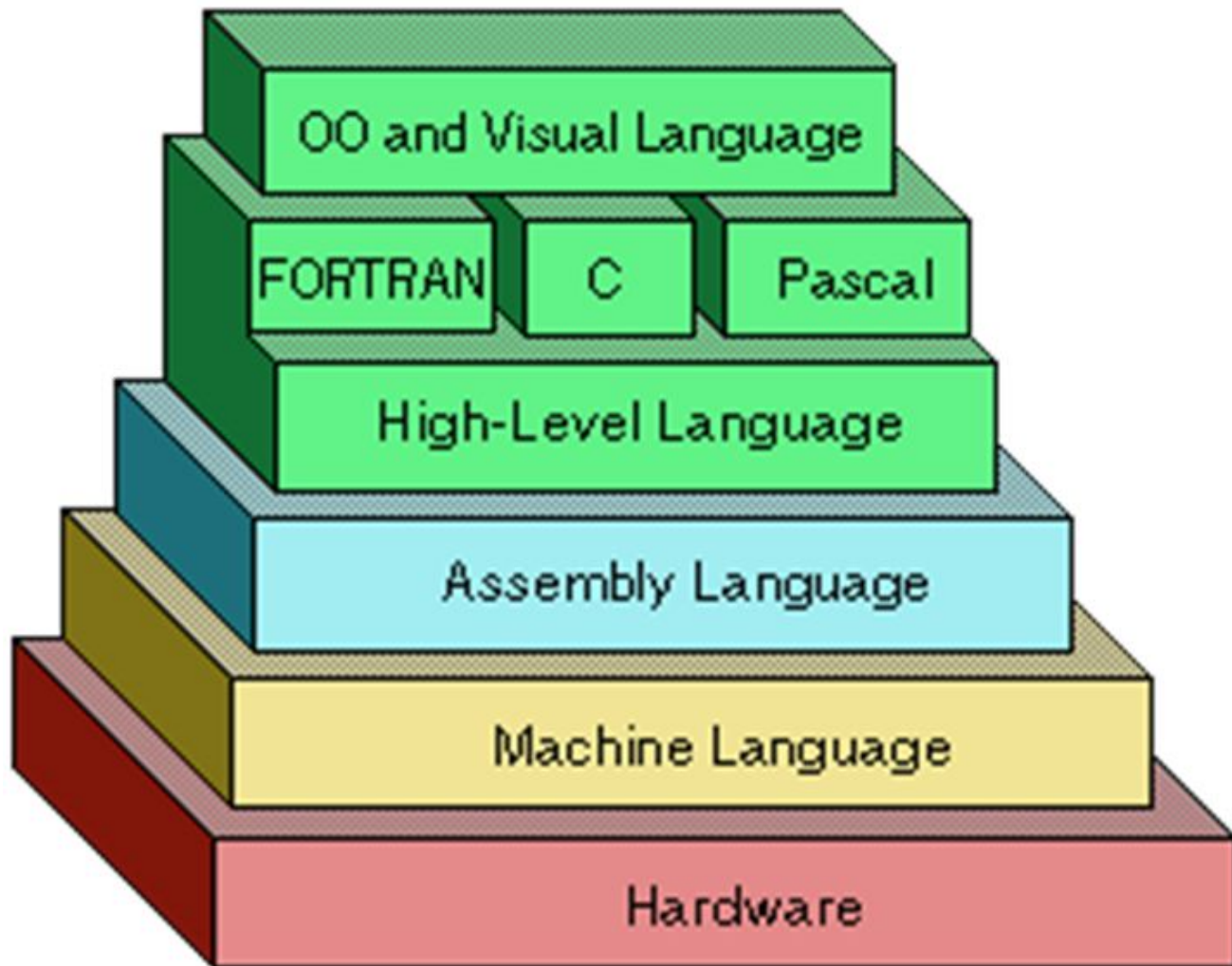
Programming Languages

Machine Language Assembly Language **High-Level Language**

The high-level languages are English-like and easy to learn and program. For example, the following is a high-level language statement that computes the area of a circle with radius 5:

```
area = 5 * 5 * 3.1415;
```

Language Hierarchy



Popular High-Level Languages

Language	Description
Ada	Named for Ada Lovelace, who worked on mechanical general-purpose computers. The Ada language was developed for the Department of Defense and is used mainly in defense projects.
BASIC	Beginner's All-purpose Symbolic Instruction Code. It was designed to be learned and used easily by beginners.
C	Developed at Bell Laboratories. C combines the power of an assembly language with the ease of use and portability of a high-level language.
C++	C++ is an object-oriented language, based on C.
C#	Pronounced "C Sharp." It is a hybrid of Java and C++ and was developed by Microsoft.
COBOL	COmm on Business Oriented Language. Used for business applications.
FORTRAN	FORmula TRANslation. Popular for scientific and mathematical applications.
Java	Developed by Sun Microsystems, now part of Oracle. It is widely used for developing platform-independent Internet applications.
Pascal	Named for Blaise Pascal, who pioneered calculating machines in the seventeenth century. It is a simple, structured, general-purpose language primarily for teaching programming.
Python	A simple general-purpose scripting language good for writing short programs.
Visual Basic	Visual Basic was developed by Microsoft and it enables the programmers to rapidly develop graphical user interfaces.

Algorithms

- An algorithm is a sequence of executable instructions with these properties.
 1. There is no ambiguity in any instruction.
 2. There is no ambiguity about which instruction is to be executed next.
 3. The description of algorithm is finite.
 4. Execution of the algorithm concludes after a finite number of steps.

Algorithms

- An execution instruction is one that can be carried out.
- For example
 - Step 1: Make a list of the odd positive integers.
 - Step 2: Compute their sum.
 - Step 3: Compute their average.
 - Step 4: Print the average.

The Programmer's Algorithms

- **An algorithm** is a finite sequence of instructions that produces a solution to a problem.
- **The programmer's algorithm:**
 - Define the problem.
 - Plan the problem solution.
 - Code the program.
 - Compile the program.
 - Run the program.
 - Test and debug the program.

Defining the Problem

- The problem must be defined in terms of:

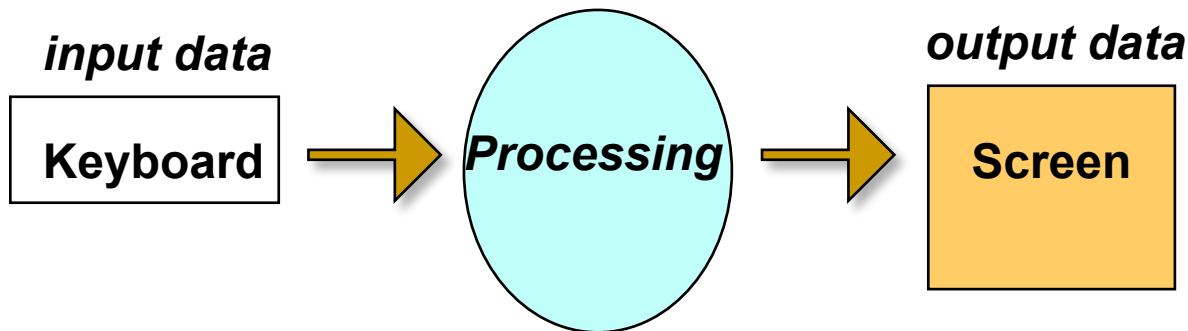
Input: Data to be processed.

Output: The expected result.

- Look for nouns in the problem statement that suggest output and input.

– **and processing:** The statements to achieve.

- Look for verbs to suggest processing steps.



Input and Output

- **Inputs**
 - Can come from many sources, such as users, files, and other programs
 - Can take on many forms, such as text, graphics, and sound
- **Outputs**
 - Can also take on many forms, such as numbers, text, graphics, sounds, or commands to other programs

Example 1

Area and Perimeter of a rectangle

- **Input**
 - Length
 - width
- **Processing**
 - $\text{Area} = \text{length} * \text{width}$
 - $\text{Perimeter} = 2 * (\text{length} + \text{width})$
- **Output**
 - Area
 - Perimeter

Example 2

Sum and Average of 5 numbers

- **Input**
 - five number x_1, x_2, x_3, x_4, x_5
- **Processing**
 - $\text{Sum} = x_1 + x_2 + x_3 + x_4 + x_5$
 - $\text{Average} = \text{Sum} / 5$
- **Output**
 - Sum
 - Average

Example 3

Area and Perimeter of a circle

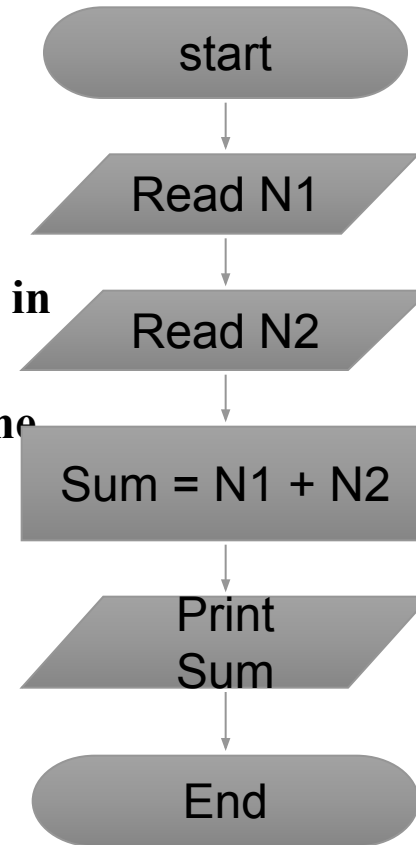
- **Input**
 - Radius
 - PI
- **Processing**
 - $\text{Area} = \text{PI} * \text{Radius} * \text{Radius}$
 - $\text{Perimeter} = 2 * \text{PI} * \text{Radius}$
- **Output**
 - Area
 - Perimeter

2-Planning the Solution

- When planning, algorithms are used to outline the solution steps using **English like statements**, called *pseudocode*.
- A *flowchart* is useful for the graphical representation of an algorithm.
 - They are drawn using rectangles, diamonds, ovals, and small circles.

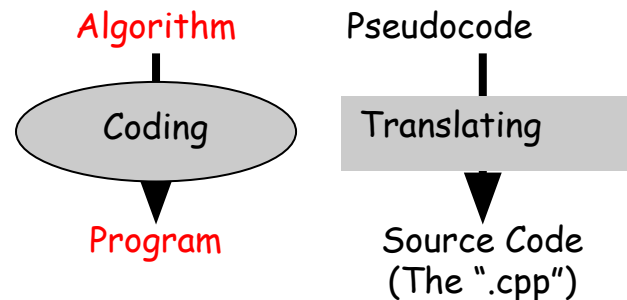
Write a Program to Print the Sum of two integer Numbers

1. **Start the program**
2. **Read the first number and save in the variable (N1)**
3. **Read the second number and save in the variable (N2)**
4. **Sum the both numbers and save the result in the variable (Sum)**
 □ $\text{Sum} = \text{N1} + \text{N2}$
5. **Print the variable (Sum)**
6. **End the program**



3-Coding the Program

- Coding is writing the program in a formal language called **Programming Language**.
 - **Programming Language** : A set of rules, symbols and special words used to write statements.
- The program is written by translating the algorithm steps into a programming language statements.
- The written program is called ***Source code*** and it is saved in a file with “.cpp” extension.

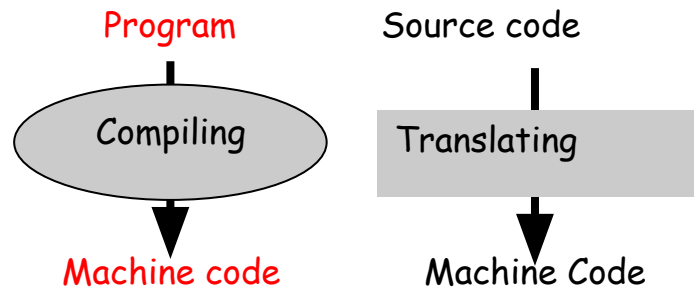


Why Coding in Programming Languages

- We write computer programs (i.e. a set of instructions) in programming languages such as C, C++, and Java.
- We use these programming languages because they are easily understood by humans
- But then how does the computer understand the instructions that we write?

4-Compiling Computer Programs

- *Computers do not understand* programs written in *programming languages* such as C, C++ and Java
- *Programs* must first be *converted into machine code* that the computer can run
- A Software that *translates* a programming language statements into machine code is called a **compiler**
- In C , **Preprocessor** program, execute automatically before the compiler translation phase begin
 - Preprocessor indicate that certain manipulations are to be performed on the program before compilation.



Programming Language Compiler

- A compiler is a software that:
 - *Checks the correctness* of the source code according to the language rules.
 - **Syntax errors** are raised if some rules were violated.
 - *Translates* the source code into a machine code if no errors were found.

5-Running The Program

- Before running..
- links the object code with the code for the missing functions to produce an executable image (with no missing pieces) , this is called **link phase** .
- If the program compiles and links correctly, a file called a.out is produced.
- Before a program can be executed, the program must first be placed in memory this called **load phase** .
- the computer, under the control of its CPU, **executes** the program one instruction at a time

6-Testing and Debugging the Program

- **Testing**

- **Be sure that the output of the program conforms with the input.**

- **There are two types of errors:**

- **Logical Errors: The program run but provides wrong output.**
 - **Runtime errors: The program stop running suddenly when asking the OS executing a non accepted statement (divide by zero, etc).**

- **Debugging**

- **Find, Understand and correct the error**

Why C++?



C++ is a general
purpose language

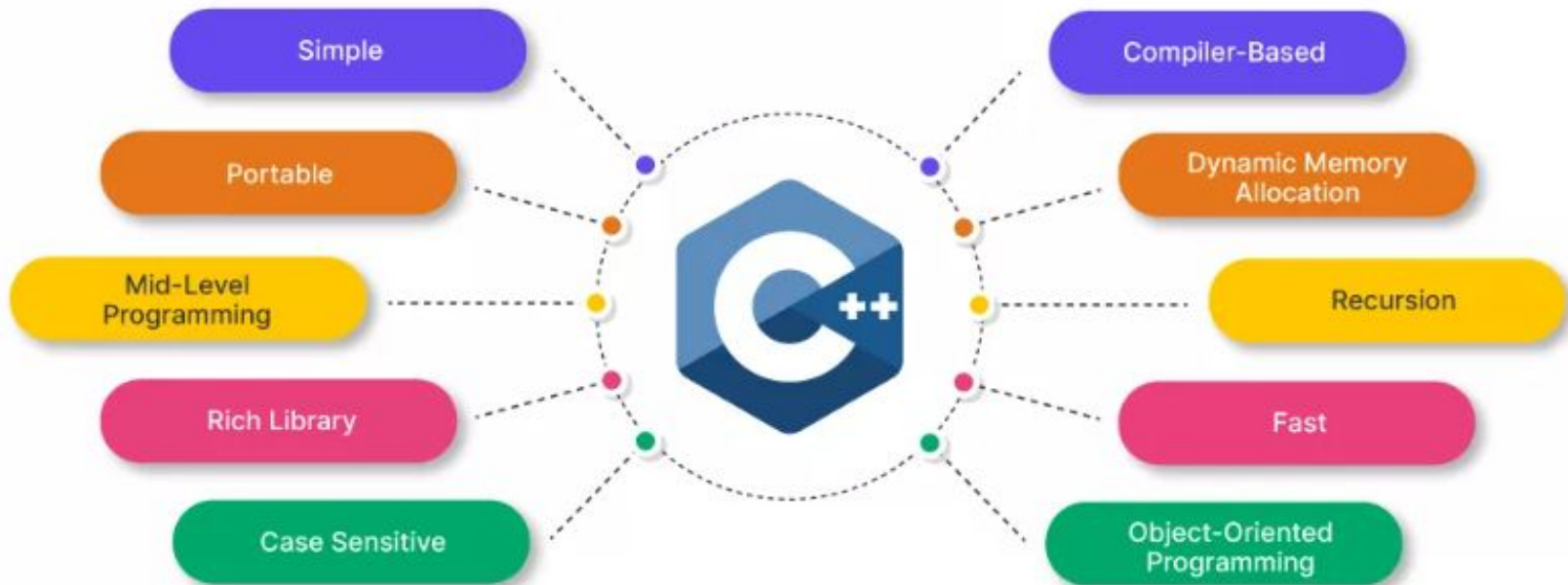


C++ emphasizes
power and
performance



C++ is popular

Features Of C++



Why C++?

- C++ is a powerful language used by millions of developers
- C++ has a reputation for being difficult
 - But it doesn't have to be
- Modern C++ is clean and simple
 - The Standard Library does a lot of work for you
- Modern C++ looks a lot like C# or Java But faster!

Companies that uses C++



Microsoft



Opera



linkedIn



Nasa



Evernote



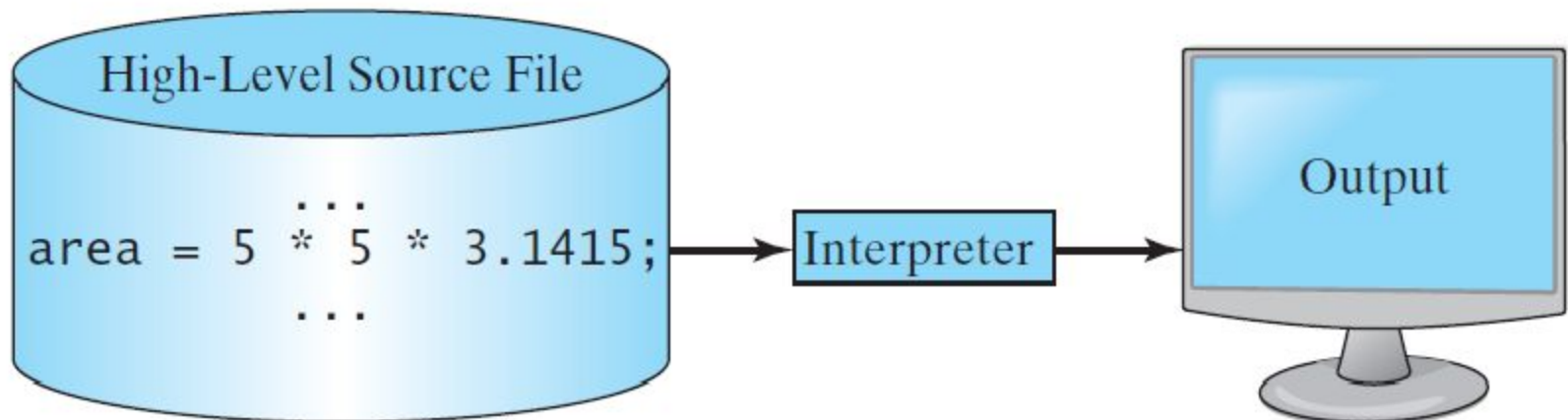
Facebook

Interpreting/Compiling Source Code

A program written in a high-level language is called a *source program* or *source code*. Because a computer cannot understand a source program, a source program must be translated into machine code for execution. The translation can be done using another programming tool called an *interpreter* or a *compiler*.

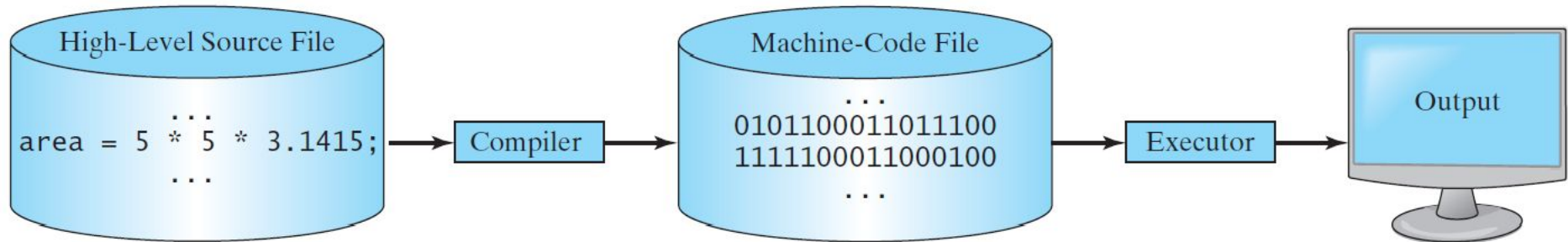
Interpreting Source Code

An interpreter reads one statement from the source code, translates it to the machine code or virtual machine code, and then executes it right away, as shown in the following figure. Note that a statement from the source code may be translated into several machine instructions.



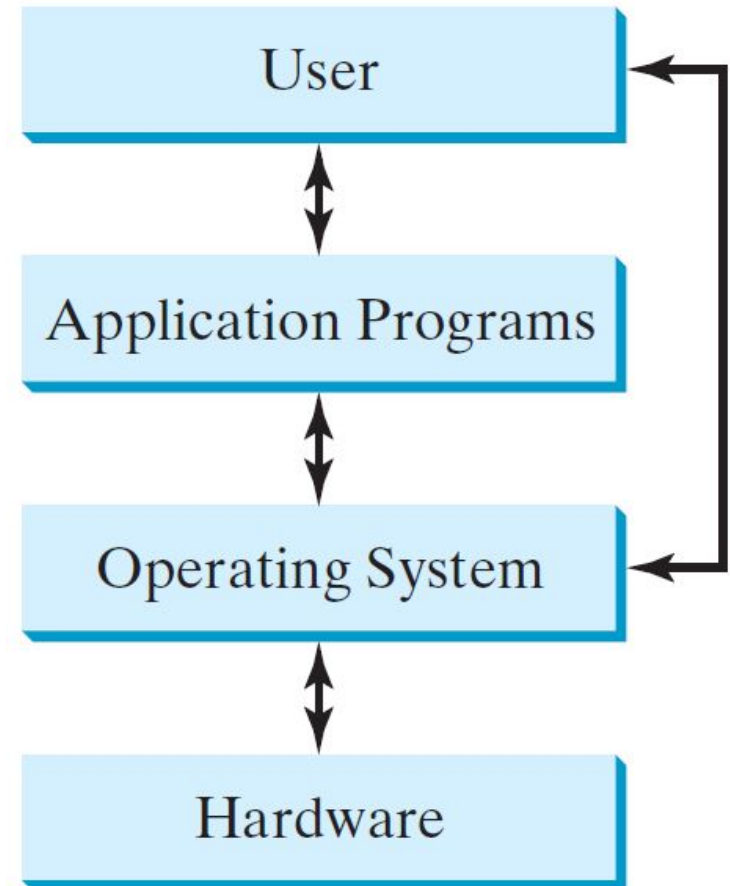
Compiling Source Code

A compiler translates the entire source code into a machine-code file, and the machine-code file is then executed, as shown in the following figure.

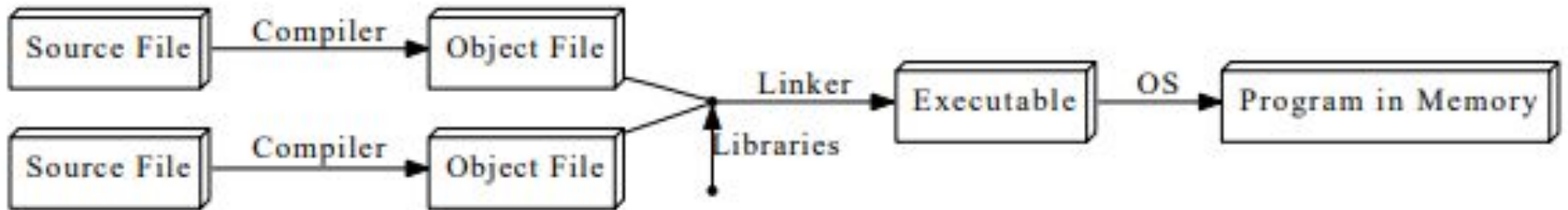


Operating Systems

The *operating system* (OS) is a program that manages and controls a computer's activities. The popular operating systems for general-purpose computers are Microsoft Windows, Mac OS, and Linux. Application programs, such as a Web browser or a word processor, cannot run unless an operating system is installed and running on the computer.



Compilation Process



Object files are intermediate files that represent an incomplete copy of the program.

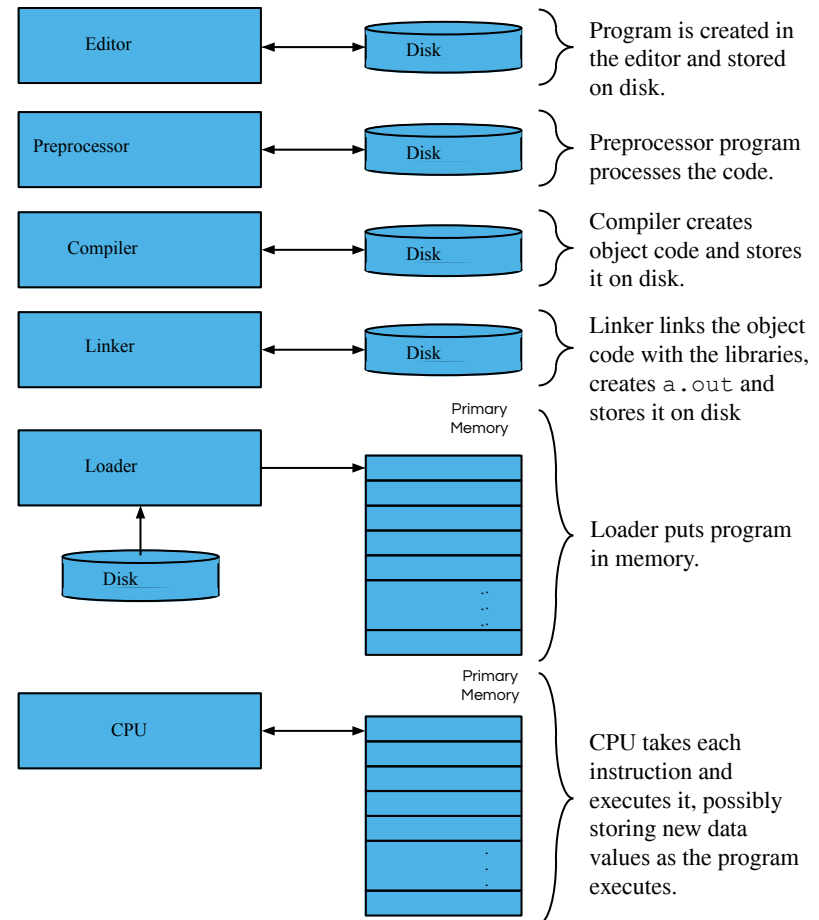
The linker takes those object files and the compiled libraries of predefined code that they rely on, fills in all the gaps, and spits out the final program, which can then be run by the operating system (OS).

1.9 Typical C++ Development Environment (Cont.)

- C++ systems generally consist of three parts: a program development environment, the language and the C++ Standard Library.
- C++ programs typically go through six phases: edit, preprocess, compile, link, load and execute.

Phases of C++ Programs:

1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute



1.9 Typical C++ Development Environment (Cont.)

- Phase 1 consists of editing a file with an *editor* program, normally known simply as an editor.
 - Type a C++ program (**source code**) using the editor.
 - Make any necessary corrections.
 - Save the program.
 - C++ source code filenames often end with the `.cpp`, `.cxx`, `.cc` or `.C` extensions, which indicate that a file contains C++ source code.

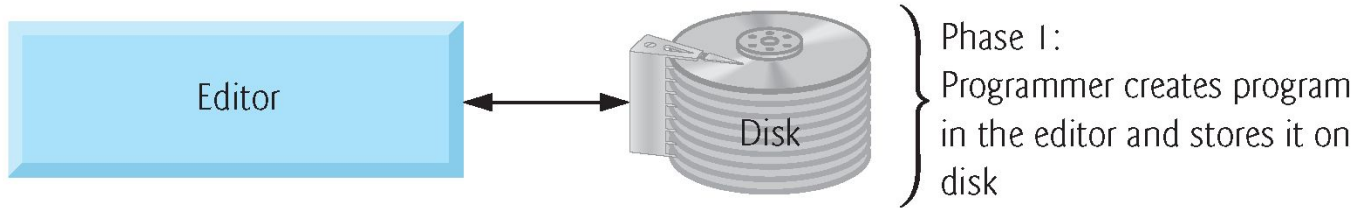


Fig. 1.6 | Typical C++ development environment—editing phase.

1.9 Typical C++ Development Environment (Cont.)

- Linux editors: `vi` and `emacs`.
- You can also use a simple text editor, such as Notepad in Windows, to write your C++ code.
- integrated development environments (IDEs)
 - Provide tools that support the software-development process, including editors for writing and editing programs and debuggers for locating **logic errors**—errors that cause programs to execute incorrectly.

1.9 Typical C++ Development Environment (Cont.)

- Popular IDEs
 - Microsoft[®] Visual Studio 2015 Community Edition
 - NetBeans
 - Eclipse
 - Apple's Xcode
 - CodeLite
 - Clion

1.9 Typical C++ Development Environment (Cont.)

- In phase 2, you give the command to **compile** the program.
 - A **preprocessor** program executes automatically before the compiler's translation phase begins (so we call preprocessing Phase 2 and compiling Phase 3).
 - The C++ preprocessor obeys commands called **preprocessing directives**, which indicate that certain manipulations are to be performed on the program before compilation.
 - These manipulations usually include (copy into the program file) other text files to be compiled, and perform various text replacements.

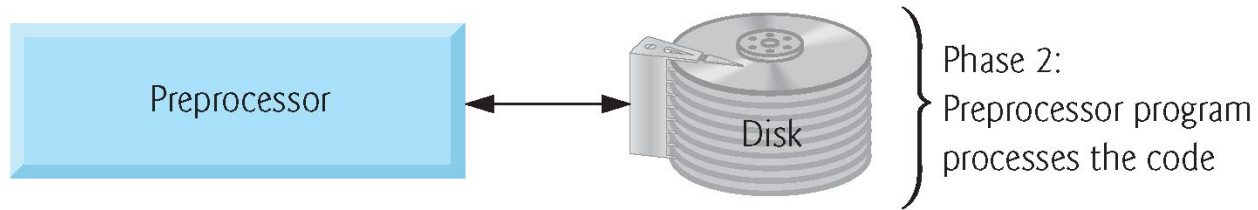


Fig. 1.7 | Typical C++ development environment—preprocessor phase.

1.9 Typical C++ Development Environment (Cont.)

- In Phase 3, the compiler translates the C++ program into machine-language code—also referred to as object code.

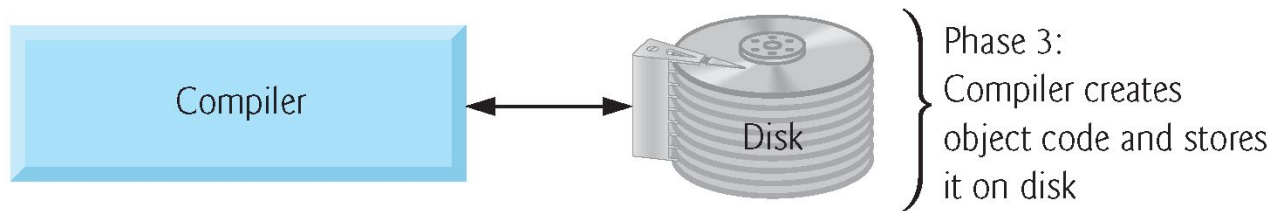


Fig. 1.8 | Typical C++ development environment—compilation phase.

1.9 Typical C++ Development Environment (Cont.)

- Phase 4 is called **linking**.
 - The object code produced by the C++ compiler typically contains “holes” due to these missing parts.
 - A **linker** links the object code with the code for the missing functions to produce an **executable program**.
 - If the program compiles and links correctly, an executable image is produced.

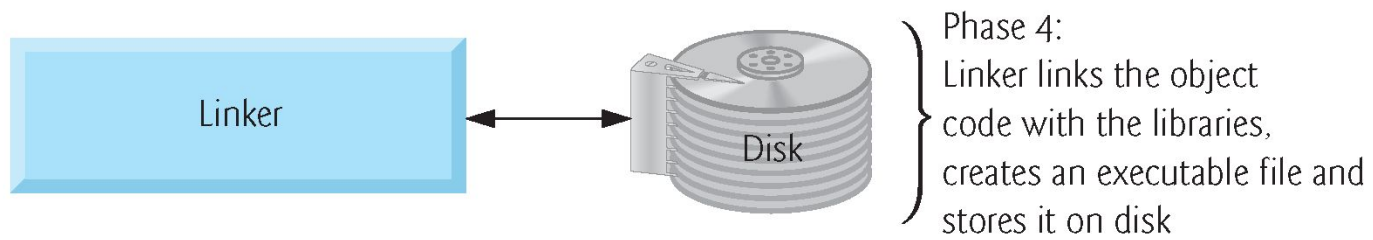


Fig. 1.9 | Typical C++ development environment—linking phase.

1.9 Typical C++ Development Environment (Cont.)

- Phase 5 is called **loading**.
 - Before a program can be executed, it must first be placed in memory.
 - This is done by the **loader**, which takes the executable image from disk and transfers it to memory.
 - Additional components from shared libraries that support the program are also loaded.

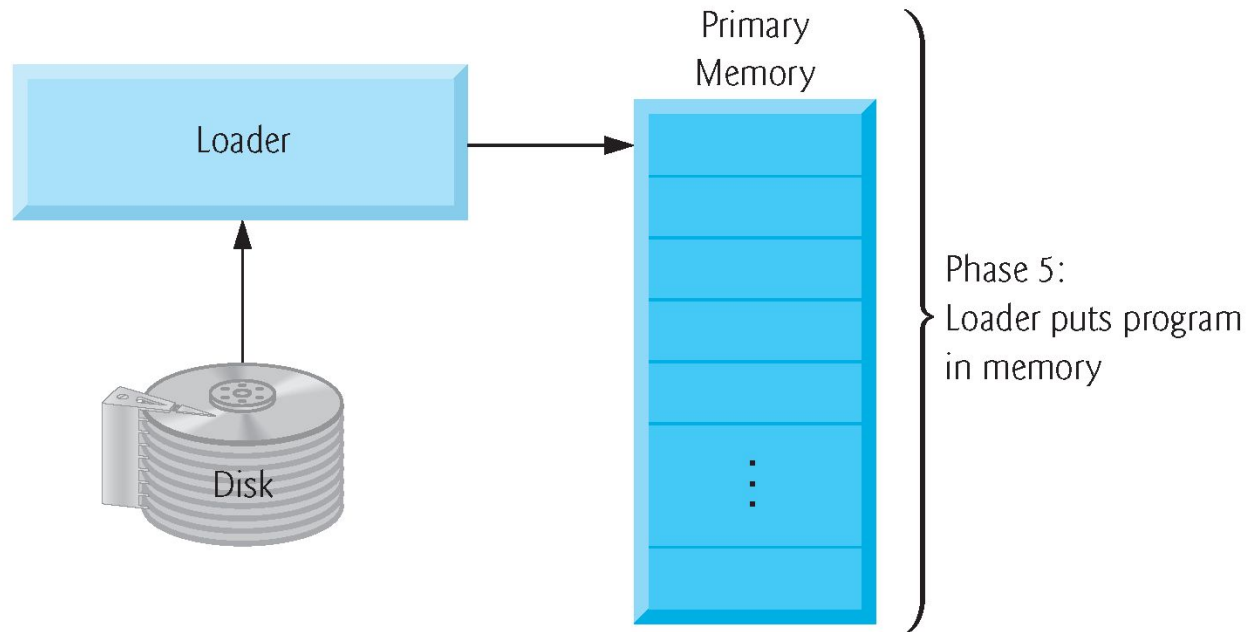


Fig. 1.10 | Typical C++ development environment—loading phase.

1.9 Typical C++ Development Environment (Cont.)

- Phase 6: Execution
 - Finally, the computer, under the control of its CPU, **executes** the program one instruction at a time.
 - Some modern computer architectures often execute several instructions in parallel.

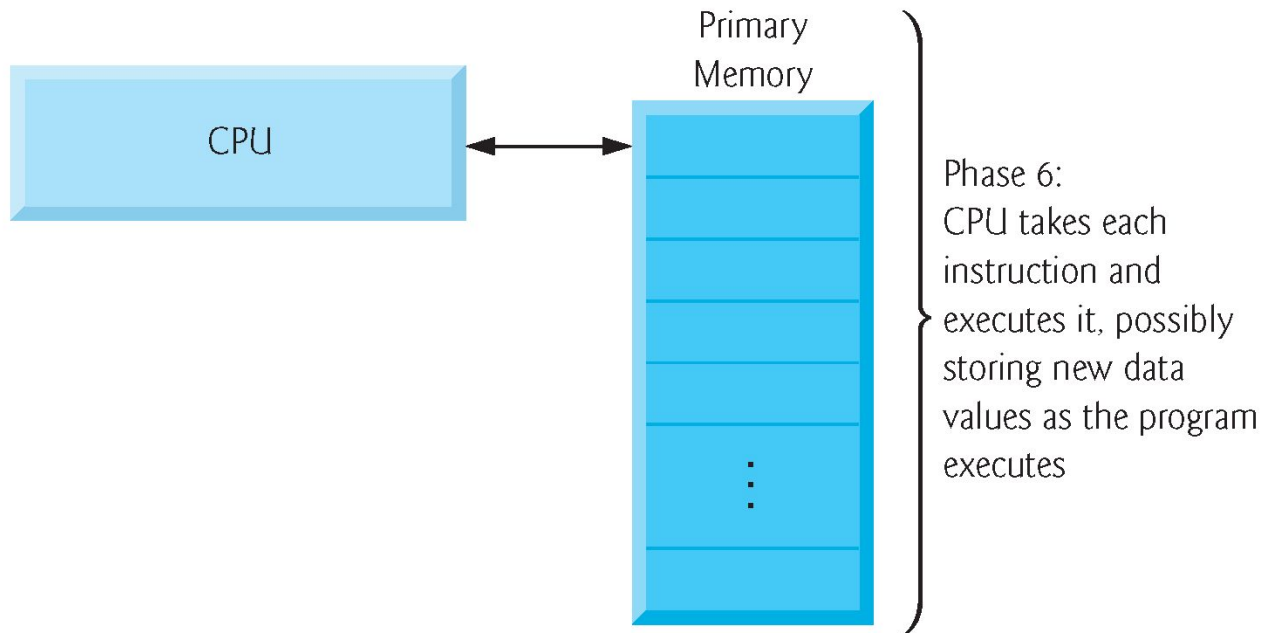


Fig. 1.11 | Typical C++ development environment—execution phase.

```

1  // Fig. 1.2: fig01_02.cpp
2  // A first program in C++
3  #include <iostream>
4
5  int main()
6  {
7      std::cout << "Welcome to C++!\n";
8
9      return 0;    // indicate that
10 }

```

Comments

Written between `/*` and `*/` or following a `//`
 Improve program readability and do not cause the computer to perform any action

of

preprocessor directive

Message to the C++ preprocessor

Lines beginning with `#` are preprocessor directives

#include <iostream> tells the preprocessor to include the contents of the file **<iostream>**, which includes input/output operations (such as printing to the screen).

C++ programs contain one or more functions, exactly one of which must be **main**

Welcome to C++!

Prints the *string* of characters contained between the

return is one a way to exit a function.

return 0, in this case, means that the program terminated normally.

including **std::cout**, the **<<** *string* **"Welcome to C++!\n"** and **(;)**, is called a *statement*.

All statements must end with a semicolon.

ger value.

y function

A Simple Program: Printing a Line of Text

- **std::cout**

- standard output stream object
- “connected” to the screen
- we need the **std::** to specify what "namespace" **cout** belongs to
 - we shall remove this prefix with **using** statements

- **<<**

- stream insertion operator
- value to the right of the operator (right operand) inserted into output stream (which is connected to the screen)

```
std::cout << " Welcome to C++!\n"
```

- ****

- escape character
- indicates that a “special” character is to be output

A Simple Program: Printing a Line of Text (II)

Escape Sequence	Description
<code>\n</code>	Newline. Position the screen cursor to the beginning of the next line.
<code>\t</code>	Horizontal tab. Move the screen cursor to the next tab stop.
<code>\r</code>	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
<code>\a</code>	Alert. Sound the system bell.
<code>\\</code>	Backslash. Used to print a backslash character.
<code>\"</code>	Double quote. Used to print a double quote character.

- There are multiple ways to print text
 - Following are more examples

```
1 // Fig. 1.4: fig01_04.cpp
2 // Printing a line with multiple statements
3 #include <iostream>
4
5 int main()
6 {
7     std::cout << "Welcome ";
8     std::cout << "to C++!\n";
9
10    return 0; // indicate that program ended successfully
11 }
```

1. Load <iostream>

2. main

2.1 Print "Welcome"

2.2 Print "to C++!"

2.3 newline

2.4 exit (return 0)

Welcome to C++!

Unless new line '**\n**' is specified, the text continues on the same line.


```
1 // Fig. 1.5: fig01_05.cpp
2 // Printing multiple lines with a single statement
3 #include <iostream>
4
5 int main()
6 {
7     std::cout << "Welcome\nto\n\nC++!\n";
8
9     return 0; // indicate that program ended successfully
10 }
```

1. Load <iostream>

2. main

2.1 Print "Welcome"

2.2 newline

2.3 Print "to"

2.4 newline

2.5 newline

2.6 Print "C++!"

2.7 newline

2.8 exit (return 0)

Welcome
to
C++!

Multiple lines can be printed with one statement

Another Simple Program:

Adding Two Integers

- Variables

- location in memory where a value can be stored for use by a program
 - must be declared with a *name* and a *data type* before they can be used
 - Must appear before variable is used
 - Some common data types are:
 - **int** - integer numbers
 - **char** - characters
 - **double** - floating point numbers
 - Example: **int myVariable;**
 - Declares a variable named **myVariable** of type **int**
- Example: **int variable1, variable2;**
- Declares two variables, each of type **int**

Another Simple Program:

Adding Two Integers

- **>>** (stream extraction operator)

- When used with **std::cin**, waits for user to input a value and stores the value in the variable to the right of the operator.
- user types number, then presses the *Enter (Return)* key to send the data to the computer
- Example:

```
int myVariable;
```

```
std::cin >> myVariable;
```

- waits for user input, then stores input in **myVariable**

- **=** (assignment operator)

- assigns value to a variable
- binary operator (has two operands)

```
sum = variable1 + variable2;
```

Addition operator

```
1 // Fig. 1.6: fig01_06.cpp
```

```
2 // Addition program
```

```
3 #include <iostream>
```

```
4
```

```
5 int main()
```

```
6 {
```

```
7     int integer1, integer2, sum;           // declaration
```

```
8
```

```
9     std::cout << "Enter first integer\n"
```

```
10    std::cin >> integer1;
```

```
11    std::cout << "Enter second integer\n"
```

```
12    std::cin >> integer2;
```

```
13    sum = integer1 + integer2;
```

```
14    std::cout << "Sum is " << sum << std::endl; // print sum
```

```
15
```

```
16    return 0; // indicate that program ended successfully
```

```
17 }
```

1. Load <iostream>

2. main

2.1 Initialize variables

integer1, integer2, and

Notice how **std::cin** is used to get user input.

2.2 Print "Enter first integer"

2.2.1 Get input

2.3 Print "Enter second

std::endl flushes the buffer and prints a newline.

2.4 Add the sum and

Variables can be output using **std::cout << variableName**

2.5 Print "Sum is"

2.5.1 Output sum

2.6 exit (return 0)

Program Output

```
Enter first integer
45
Enter second integer
72
Sum is 117
```

```
1 // Fig. 2.5: fig02_05.cpp
2 // Addition program that displays the sum of two integers.
3 #include <iostream> // enables program to perform input and output
4
5 // function main begins program execution
6 int main() {
7     // declaring and initializing variables
8     int number1{0}; // first integer to add (initialized to 0)
9     int number2{0}; // second integer to add (initialized to 0)
10    int sum{0}; // sum of number1 and number2 (initialized to 0)
11
12    std::cout << "Enter first integer: "; // prompt user for data
13    std::cin >> number1; // read first integer from user into number1
14
15    std::cout << "Enter second integer: "; // prompt user for data
16    std::cin >> number2; // read second integer from user into number2
17
18    sum = number1 + number2; // add the numbers; store result in sum
19
20    std::cout << "Sum is " << sum << std::endl; // display sum; end line
21 }
```

Fig. 2.5 | Addition program that displays the sum of two integers.